تعمیر خودکار برنامه آگاه از قیود معماری با استفاده از گراف‌های مخزن و بازخورد اجرا :ArchGuard-APR

مجید فیض دیلمی
دانشجوی کارشناسی ارشد مهندسی نرم‌افزار
گروه مهندسی کامپیوتر، واحد لاهیجان، دانشگاه آزاد اسلامی، لاهیجان، ایران
مسئول مکاتبات: majid.feiz@iau.ir · ORCID: https://orcid.org/0009-0006-8439-1839

□□□□ □□□□ □□□□□ — □□□□□□□ □□□□

چکیده

در مقیاس مخزن باید افزون بر (Automated Program Repair) تعمیر خودکار برنامه رفتار موضعی، ساختار سامانه را نیز حفظ کند .یک تغییر پیشنهادی ممکن است آزمونی محدود را بگذراند، اما مرز وابستگی را نقض کند، مسیر ناایمنی را تغییر دهد، یا بر زمینه‌ای ناقص نمونه‌ای پژوهشی برای تعمیر آگاه از معماری است که تحلیل ArchGuard-APR .تکیه کند قطعی مخزن، گراف‌های تغییرناپذیر، پروفایل‌های صریح معماری، بازیابی زمینه مبتنی بر گراف، هماهنگی نقش‌محور، کنترل سیاست وصله، بازخورد اجرای ایزوله، محدودیت هزینه و شواهد استاندارد و عاری از کد منبع را یکپارچه می‌کند .ارزیابی انجام‌شده یک مطالعه زوجی اکتشافی امکان‌پذیری است و نه آزمایشی تأییدی درباره اثربخشی .از شش اجرای پیش از تماس با ارائه‌دهنده دچار black-4 برنامه‌ریزی‌شده، چهار اجرا کامل شد و دو وضعیت هر ArchGuard ریزش زیرساختی شدند .در دو وظیفه زوجی معتبر، نرخ موفقیت خط پایه و دو ۰ از ۲ بود و هر چهار خروجی کامل‌شده »وصله نامعتبر «طبقه‌بندی شدند .چهار اجرا توکن ورودی و ۲،۷۳۷ توکن خروجی، در مجموع ۳۸،۱۶۸ توکن، با هزینه ۳۵،۴۳۱ دلار مصرف کردند .تفاوتی در اثربخشی مشاهده نشد و اندازه نمونه برای استنباط ۰,۱۲۹۶۳۲۵ آماری کافی نیست .نتیجه اصلی، امکان‌پذیری مهندسی خط لوله و ضرورت بهبود اعتبار وصله و پایداری زیرساخت است.

واژگان کلیدی— تعمیر خودکار برنامه؛ معماری نرم‌افزار؛ گراف مخزن؛ قید معماری؛ مدل زبانی بزرگ؛ سامانه چندعاملی؛ بازخورد اجرا؛ بازتولیدپذیری.

۱. مقدمه

تعمیر خودکار برنامه می‌کوشد تغییراتی تولید کند که نقص نرم‌افزار را برطرف سازند .در وظایف سطح مخزن، مسئله به یک تابع محدود نیست :ابزار باید کد مرتبط را بیابد، سازمان پروژه را رعایت کند، وصله‌ای مجاز بسازد و رفتار را در محیطی کنترل‌شده اعتبارسنجی کند . عامل‌های مبتنی بر مدل زبانی امکان برنامه‌ریزی و کار با ابزارها را افزایش می‌دهند، اما مرزهای شکست بیشتری نیز پدید می‌آورند .پاسخ متنی محتمل لزوماً وصله‌ای قابل تجزیه نیست؛ .وصله قابل اعمال نیز الزاماً ایمن یا منطبق با معماری نیست

این مرزها را اشیای صریح پژوهش می‌داند .هسته دامنه و کاربرد از ArchGuard-APR سازگارسازهای زیرساختی و فروشنده مستقل است .منبع مخزن ابتدا به تصویر لحظه‌ای اعتبارسنجی‌شده تبدیل می‌شود؛ ساخت گراف تنها همان تصویر را مصرف می‌کند؛ ارزیابی معماری نیز فقط گراف معتبر و پروفایل نسخه‌دار را می‌پذیرد .بازیابی زمینه بودجه‌دار است و .بازرسی وصله و اجرا زیر سیاست‌های ایمنی، ایزوله‌سازی و ثبت قطعی شواهد انجام می‌شود

مقاله دو سهم جدا دارد .سهم مهندسی، نمونه‌ای پیاده‌سازی‌شده از تحلیل مخزن، قیود معماری، بازیابی، نقش‌های تعمیر، اعتبارسنجی وصله و بازخورد اجراست .شواهد علمی محدودترند : چهار خروجی نامعتبر و دو ریزش زیرساختی .بنابراین مقاله ادعای برتری ندارد، آزمون .معنادار استنباطی انجام نمی‌دهد و نتایج منفی را حذف نمی‌کند

۲. مبانی و انگیزه

روش‌های تولید و اعتبارسنجی، نامزدها را می‌سازند و در برابر نشانه‌ای از درستی می‌آزمایند . در مقیاس مخزن، زمینه مرتبط ممکن است از مرز ماژول عبور کند، محیط ساخت مستقل از وصله شکست بخورد و مجموعه آزمون مشخصاتی ناقص باشد .معماری بعد دیگری از درستی را معرفی می‌کند :مرزهای وابستگی و مالکیت می‌توانند قصد نگهداشت‌پذیری را بیان کنند که .آزمون رفتاری آن را نمی‌سنجد

۲.۱. خط لوله تعمیر و معنای پیامدها

از تحلیل معنایی SemFix جست‌وجوی مبتنی بر آزمون را به کار می‌گیرد]4[؛ GenProg رتبه‌بندی نامزدها را Prophet .از سنتز نمادین چندخطی استفاده می‌کند]7[Angelix و [5] زمینه شیءگرا را وارد انتخاب می‌کند]10 .[این Elixir از وصله‌های انسانی می‌آموزد]8 [و تفاوت‌ها نشان می‌دهند سازوکار پیشنهاد وصله از داوری پذیرش آن جداست .مطالعات .نیز نشان داده‌اند وصله آزمون‌گذر می‌تواند نادرست بماند]9[Defects4J

در این مقاله »نامزد «هر ویرایش پیشنهادی است؛ »وصله معتبر «قرارداد قالب، نحو، مسیر، اندازه و سیاست را رعایت می‌کند؛ و موفقیت تعمیر افزون بر قابلیت اعمال، به نتیجه ارزیابی وابسته است .پاسخ ارائه‌دهنده که به وصله مجاز تبدیل نشود یک اجرای علمی با طبقه‌بندی

است .شکست کسب یا اعتبارسنجی مخزن پیش از ارسال به ارائه‌دهنده، پیامد `invalid_patch` .مدل نیست و ریزش زیرساختی ثبت می‌شود

۲.۲. تعمیر سطح مخزن و زمینه

مجموعه‌ای متناظر BugsInPy مجموعه‌ای کنترل‌شده از خطاهای جاوا [6] و Defects4J مسئله را به رفع درخواست واقعی در مخزن و SWE-bench .برای پایتون فراهم می‌کند [14] محیط اجرای آن گسترش می‌دهد [20] .[این بنچمارک‌ها از نظر زبان، وابستگی، ساخت وظیفه و محیط یکسان نیستند .بازیابی افزوده، شواهد بازیابی‌شده را از پارامترهای مدل جدا می‌کند [15]؛ نیز سودمندی روابط صریح را GraphCodeBERT [16] نمایش‌های گرافی برنامه [11] و نشان می‌دهند .گراف ایستای این سامانه با نمایش‌های یادگرفته‌شده یکسان نیست؛ هدف آن انتخاب .محدود، قابل ممیزی و تکرارپذیر زمینه است

۲.۳. معماری و انطباق

معماری نرم‌افزار با عناصر، روابط و منطق طراحی توصیف می‌شود [1] .[مدل‌های بازتابی پروفایل ArchGuard-APR فاصله میان مدل سطح بالا و ساختار منبع را می‌سنجند [2] .[در بازبینی‌شده انسانی، انتساب ماژول‌ها و وابستگی‌های مجاز را تعیین می‌کند .سامانه معماری را .استنباط نمی‌کند و در برابر ماژول مبهم یا بدون انتساب، بسته و محافظه‌کارانه شکست می‌خورد

۳. کارهای مرتبط

.گستره تعمیر رفتاری و انواع داور را صورت‌بندی می‌کند [3] Monperrus مرور ترجمه عصبی آگاه از زمینه را به کار می‌گیرد [13] و ارزیابی مدل‌های بزرگ CoCoNuT تفاوت ابزار و RepairThemAll .از ظرفیت آن‌ها برای تولید وصله گزارش می‌دهد [19] بنچمارک را در مقیاس بزرگ مستند کرده است [12] .[این مطالعات اهمیت فضای نامزد، زمینه .را ثابت نمی‌کنند ArchGuard-APR و اعتبارسنجی را روشن می‌کنند، اما اثربخشی

جست‌وجوی AutoCodeRover ،حلقه ابزارمحور RepairAgent [21] ،در تعمیر مخزن .استخراج قصد و نقش بازبین [23] را بررسی می‌کنند SpecRover ساختاریافته [22] و از بازخورد زبانی استفاده Reflexion استدلال و عمل را درهم می‌آمیزد [17] و ReAct می‌کند [18] .[سامانه حاضر از این ایده‌ها برای جداسازی نقش‌ها الهام می‌گیرد، ولی نوآوری یا تقدم تاریخی فراگیر ادعا نمی‌کند .تمایز آن در ترکیب پروفایل معماری صریح با شواهد قطعی، .کنترل ایمنی و حسابداری اجرای منبع‌آزاد است

۴. پرسش‌های پژوهش

RQ1: آیا قیود معماری، گراف مخزن، بازیابی زمینه و بازخورد اجرا را می‌توان در خط لوله‌ای بازتولیدپذیر یکپارچه کرد؟ **RQ2:** چه محدودیت‌های عملیاتی در اجرای وظایف واجد شرایط پدیدار می‌شود؟ **RQ3:** چه پیامدها، مصرف توکن و هزینه‌ای مشاهده می‌شود؟ هیچ پرسشی برتری را پیش‌فرض نمی‌گیرد.

۵. طراحی ArchGuard-APR

ورودی‌های خط فرمان، خدمات کاربرد را از راه قراردادهای دامنه فراخوانی می‌کنند. جریان Git، سازگارسازها عملیات ذخیره مصنوعات، ظرف اجرا و ارائه‌دهنده را پیاده می‌کنند. مخزن به تصویر معتبر، تصویر به گراف، گراف و پروفایل به گزارش داده یک‌طرفه است: معماری، و گزارش و پرس‌وجوی زمینه به بسته محدود تعمیر تبدیل می‌شوند. هیچ مرحله پایین‌دستی برای تکمیل اطلاعات، کد هدف را دوباره نمی‌خواند.

۵.۱. مسئولیت، اعتماد و جریان داده

مرزهای اعتماد میان کد میزبان، مخزن هدف، ارائه‌دهنده و محیط اجرای ایزوله صریح‌اند. شناسه‌ها و اثرانگشت‌ها در هر تبدیل ثبت می‌شوند. کنترل هزینه پیش از آغاز تماس ظرفیت لازم را رزرو می‌کند و پس از دریافت مصرف واقعی، رزرو را آزاد و هزینه مشاهده‌شده را دقیقاً یک‌بار ثبت می‌کند. هیچ کلید یا پاسخ مدل در بسته مقاله قرار ندارد.

۶. تحلیل مخزن و ساخت گراف

تحلیل‌گر AST بدون واردکردن یا اجرای کد هدف، فایل‌های مجاز را با import، استاندارد تولید می‌کند. گراف شامل ماژول، تعریف و رابطه دربرگیری است. JSON مرتب‌سازی پایدار و استاندارد، مصنوعات یکسان را در مکان‌های متفاوت همسان نگه می‌دارد. ریشه محافظت‌شده مانع نوشتن خروجی درون مخزن تحلیل‌شده می‌شود.

۶.۱. ناوردای تصویر و گراف

گراف فقط از تصویر اعتبارسنجی‌شده ساخته می‌شود. مسیر خارج از ریشه، پیوند نمادین ناایمن، فایل تغییرکرده یا اثرانگشت ناسازگار باعث توقف می‌شود. این سخت‌گیری پوشش gitlink، و کد تولیدی را تضمین نمی‌کند؛ چنین مواردی در تهدیدهای اعتبار باقی reflection، تحلیل پویا می‌مانند.

۷. مدل قید معماری

نسخه‌دار، قواعد انتساب و وابستگی را به بازنمایی استاندارد کامپایل می‌کند . YAML پروفایل چرخه، وابستگی ممنوع و ماژول نامشخص با شناسه پایدار گزارش می‌شوند .پروفایل حاصل قضاوت انسانی است، نه حقیقت استخراج‌شده خودکار؛ ازاین‌رو منشأ، نسخه و بازبینی آن بخشی .از شواهد است

۸. گردش کار تعمیر چندعاملی

هماهنگ‌کننده، تحلیل، بازیابی، تولید، اعتبارسنجی و ارزیابی را با حالت‌های نوع‌دار پیش می‌برد. نقش‌ها مسئولیت را جدا می‌کنند، اما به معنای استقلال علمی چند مدل نیستند .تماس ارائه‌دهنده تنها پس از تأیید پیکربندی منجمد، پذیرش هزینه و گذر از کنترل امنیتی ممکن است .وصله نامعتبر به .نتیجه پایانی صریح تبدیل می‌شود و با تصحیح خاموش قالب، موفق جلوه داده نمی‌شود

۸.۱. چرخه نامزد و حسابداری

هر تماس شناسه یکتا، وضعیت شروع و پایان، مصرف و هزینه دارد .تلاش مجدد در آزمایش نهایی صفر بود .شمارش فراخوانی و هزینه به شرط تکمیل مصرف دقیقاً یک‌بار انجام می‌شود . .طبقه‌بندی زیرساختی پیش از ارائه‌دهنده از طبقه‌بندی علمی پس از پاسخ جداست

۹. بازیابی زمینه و بازخورد اجرا

پرس‌وجوی زمینه همسایگی ماژول و نماد را با سقف تعداد و اندازه انتخاب می‌کند و منشأ هر می‌نویسد .اجرای وصله پس از کنترل مسیر و محتوا در محیط محدود manifest قطعه را در ،انجام می‌شود .بازخورد شامل وضعیت‌های استاندارد و اثرانگشت است و متن حساس، کد منبع .یا پاسخ خام را منتشر نمی‌کند prompt

۱۰. امنیت، ایزوله‌سازی و بازتولیدپذیری

خط لوله در برابر خروج از مسیر، نشت پاسخ طلایی، دسترسی شبکه نامجاز و نوشتن در مخزن حافظه، فرایند، زمان و خروجی، دامنه خسارت را کاهش ،CPU هدف بسته عمل می‌کند .سقف می‌دهد ولی خطر را صفر نمی‌کند .شواهد با ترتیب پایدار، زمان‌های حذف‌شده از محتوای .ثبت می‌شوند SHA-256 استاندارد و اثرانگشت

۱۱. روش‌شناسی آزمایش

در مجموع شش اجرا، ،ArchGuard پروتکل اکتشافی سه وظیفه را در دو شرط خط پایه و برنامه‌ریزی کرد .ترتیب، جمعیت، مدل، پیکربندی ارائه‌دهنده و بودجه پیش از مشاهده نتیجه .منجمد شدند .معیار موفقیت پروژه‌محور و سیاست وصله سخت‌گیرانه بود

۱۱.۱. احراز صلاحیت بدون مشاهده پیامد

وظایف بر اساس مجوز، دسترسی به بازبینی تغییرناپذیر، قابلیت اجرای ارزیاب و محدودیت منابع انتخاب شدند. این فرایند از وصله طلایی برای هدایت تولید استفاده نکرد. تنها دو زوج نتیجه علمی کامل ساختند؛ یک پروژه در نتایج معتبر نمایندگی شد PySnooper.

۱۱.۲. ساخت تیمار و انجماد

بسته ArchGuard خط پایه زمینه مسئله و مخزن را بدون گزارش معماری دریافت کرد. شرط زمینه و بازخورد قیود معماری را نیز دریافت کرد. هر دو از مدل `gpt-5.4-2026-03-05` و سقف‌های یکسان ارائه‌دهنده استفاده کردند. این گزارش هیچ اجرای تازه‌ای ایجاد نمی‌کند و تنها شواهد منجمد را بازگو می‌کند.

۱۱.۳. سیاست طبقه‌بندی و ریزش

دو `invalid_patch`، چهار پاسخ دریافت‌شده به دلیل عبور نکردن از قرارداد وصله شدند. پیش از تماس `validation_failure` و با رده checkout در مرحله black-4 اجرای ارائه‌دهنده متوقف شدند. اثرانگشت تشخیص `71f60e3969313f139c493cad31ee20640fd6c2870d70411debdd5ab1956994e1` است. این دو مورد شکست مدل یا تعمیر نیستند.

۱۱.۴. کنترل‌های بازتولیدپذیری

اثرانگشت جمعیت `421f3129ef6bee5883c41d05f087d6faff5b2a3de3b2ea543b4d27ade1488886`، پروتکل `eae1920ce71c025d2d94096798daf8b14df098249743631efe9a0aa7a541fbe8` و پیکربندی ارائه‌دهنده `f5dc6fb5b8bf54c7cf1b58b990fa3db44073500df9903e3a7d93cbeb1ad92c2d` است. ثبت نسخه‌ها، ترتیب اجرا و هزینه، ممیزی را ممکن می‌کند؛ بازتولید دقیق رفتار احتمالی ارائه‌دهنده تضمین نمی‌شود.

۱۲. نتایج اکتشافی

جدول ۱. خلاصه نتایج و هزینه. خط پایه: ۲ زوج معتبر، موفقیت ۱٫۰، ۲ وصله نامعتبر، ۲ توکن ورودی، ۱٬۰۳۷ توکن خروجی و هزینه ۰٫۰۳۵۱۰۷۵ دلار. ArchGuard: ۷٬۸۲۱ زوج معتبر، موفقیت ۱٫۰، ۲ وصله نامعتبر، ۲۷٬۶۱۰ توکن ورودی، ۱٬۷۰۰ توکن خروجی و

هزینه ۰٫۰۹۴۵۲۵ دلار .مجموع :۲ زوج معتبر، موفقیت ۰/۴، ۴ وصله نامعتبر، ۳۵،۴۳۱ توکن ورودی، ۲،۷۳۷ توکن خروجی و هزینه ۰٫۱۲۹۶۳۲۵ دلار.

از شش اجرای برنامه‌ریزی‌شده، چهار اجرا کامل و دو اجرا دچار ریزش شدند؛ نرخ ریزش زیرساختی ۳۳٫۳ درصد است .هر دو شرط ۰ از ۲ موفقیت داشتند و هر چهار خروجی کامل‌شده وصله نامعتبر بود .بنابراین تفاوت اثربخشی مشاهده نشد، اما برابری دو روش نیز اثبات در این نمونه کوچک ورودی و هزینه بیشتری مصرف کرد؛ این ArchGuard نمی‌شود .شرط مشاهده قابل تعمیم نیست.

۱۲.۱. تفسیر توصیفی

تنها دو زوج معتبر و صفر زوج ناهمخوان موفق وجود دارد .آزمون معناداری یا برآورد اثر پایدار با این داده موجه نیست .این ارزیابی باید امکان‌پذیری اکتشافی تلقی شود، نه ارزیابی تأییدی اثربخشی.

۱۳. تحلیل شکست

شواهد ثابت می‌کنند چهار خروجی در مرز سیاست وصله رد شدند، اما علت یگانه‌ای را اثبات نمی‌کنند .قالب، دستور زبان وصله، سیاست مسیر، کفایت زمینه، یکپارچه‌سازی و رفتار مدل همگی سازوکارهای محتمل‌اند .نسبت دادن قطعی شکست به یکی از آن‌ها بدون شواهد بیشتر ناروا‌ست.

۱۳.۱. مرز وصله نامعتبر

اعتبارسنجی سخت‌گیرانه، ایمنی و وضوح شواهد را افزایش می‌دهد و هم‌زمان می‌تواند بازده ظاهری را کاهش دهد .پژوهش بعدی باید تشخیص منبع‌آزاد دقیق‌تری برای مراحل تجزیه و سیاست فراهم کند، بدون آنکه پاسخ خام یا جزئیات حساس را منتشر سازد.

۱۳.۲. ریزش checkout

متوقف شدند. checkout پیش از تماس با ارائه‌دهنده در اعتبارسنجی black-4 هر دو وضعیت آن‌ها در مخرج نتایج تعمیر قرار نمی‌گیرند و تنها به‌عنوان ریزش زیرساختی گزارش می‌شوند . این رخداد نیاز به آزمون بیشتر کسب مخزن و ناوردای بازبینی را نشان می‌دهد، نه ضعف مدل را.

۱۴. بحث

پیاده‌سازی نشان می‌دهد می‌توان قیود معماری را با خط لوله‌ای قابل ممیزی ترکیب کرد. از نظر یکپارچه‌سازی مهندسی RQ1 بااین‌حال، داده‌ها هیچ برتری تعمیر را نشان نمی‌دهند. اعتبار وصله و پایداری زیرساخت را محدودیت‌های اصلی می‌یابد؛ و RQ2 پشتیبانی می‌شود؛ چهار وصله نامعتبر، ۳۸،۱۶۸ توکن و هزینه ۰,۱۲۹۶۳۲۵ دلار را ثبت می‌کند RQ3.

با زمینه ورودی بزرگ‌تر سازگار است، اما دو زوج برای ArchGuard افزایش هزینه شرط نتیجه‌گیری درباره نسبت هزینه و فایده کافی نیست. ارزش گراف و قید نیز از نرخ موفقیت صفر checkout، قابل جداکردن نیست. مطالعه بعدی باید پس از رفع مشکلات اعتبار وصله و جمعیت چندپروژه‌ای بزرگ‌تری را به‌صورت آینده‌نگر طراحی کند.

۱۵. تهدیدهای اعتبار

اعتبار سازه به تعریف سخت‌گیرانه و پروژه‌محور موفقیت وابسته است. اعتبار درونی با علت نامعلوم وصله‌های نامعتبر و وابستگی محیطی محدود می‌شود. اعتبار بیرونی تنها بر دو زوج از یک پروژه تکیه دارد. اعتبار نتیجه‌گیری با نبود تعمیر موفق و صفر زوج ناهمخوان، هرگونه استنباط یا ادعای برتری را رد می‌کند. ریزش زیرساختی ۳۳,۳ درصد است. نتایج به مدل، ارائه‌دهنده، زمان و پیکربندی وابسته‌اند؛ تحلیل ایستا رفتار پویا را کامل نمی‌بیند؛ پروفایل به قضاوت انسانی نیاز دارد؛ و ایزوله‌سازی همه خطرها را حذف نمی‌کند.

۱۶. ملاحظات اخلاقی و بازتولیدپذیری

نتایج منفی و هزینه‌ها حفظ شده‌اند. تنها منابع تأییدشده به کار رفته‌اند. هیچ کد هدف، وصله یا پاسخ ارائه‌دهنده در بسته مقاله نیست. مسئولیت تفسیر، افشا و ارسال prompt ،طلایی، کلید بر عهده نویسنده انسانی است. اثرانگشت نتیجه نهایی `761d9853f6ecfb4dafdaad81de781c4d49108b8b82912aeec509e173c7d56c70` است.

۱۷. نتیجه‌گیری و کار آینده

ArchGuard-APR، قیود معماری، گراف مخزن، بازیابی زمینه، هماهنگی نقش‌محور، اعتبارسنجی وصله و بازخورد ایزوله را در زیرساختی قابل ممیزی گرد هم می‌آورد. فاز ۱۳ امکان ترکیب این اجزا را نشان داد، ولی شاهدی برای مزیت تعمیر فراهم نکرد. اولویت آینده، و سپس ارزیابی آینده‌نگر بزرگ‌تر است checkout تولید وصله معتبر، تشخیص بهتر، پایداری.

دسترس‌پذیری داده‌ها و مصنوعات

و داده‌های انتشار research/results/phase-13/ داده‌های استاندارد و عاری از کد منبع در قرار دارند .مخزن شامل جدول‌ها، شکل‌ها، اثرانگشت‌ها و آزمون‌های paper/data/ در .راستی‌آزمایی است .مقاله ارسال یا پذیرفته نشده و محل انتشار هنوز تعیین نشده است

مشارکت نویسنده

نویسنده به‌تنهایی مسئول مفهوم‌پردازی، طراحی روش پژوهش، توسعه نرم‌افزار، اعتبارسنجی، .انجام بررسی‌ها، مدیریت داده‌ها، تحلیل رسمی، مصورسازی و آماده‌سازی مقاله بوده است

حمایت مالی

.این پژوهش هیچ‌گونه حمایت مالی خارجی دریافت نکرده است

تعارض منافع

.نویسنده اعلام می‌کند که هیچ‌گونه تعارض منافعی وجود ندارد

منابع

[1] D. E. Perry and A. L. Wolf, “Foundations for the Study of Software Architecture,” ACM SIGSOFT Software Engineering Notes, 1992, doi:10.1145/141874.141884.

[2] G. C. Murphy, D. Notkin, and K. J. Sullivan, “Software Reflexion Models: Bridging the Gap Between Source and High-Level Models,” ACM SIGSOFT FSE, 1995, doi:10.1145/222132.222136.

[3] M. Monperrus, “Automatic Software Repair: A Bibliography,” ACM Computing Surveys, 2018, doi:10.1145/3105906.

[4] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “GenProg: A Generic Method for Automatic Software Repair,” IEEE TSE, 2012, doi:10.1109/TSE.2011.104.

[5] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, “SemFix: Program Repair via Semantic Analysis,” ICSE, 2013, doi:10.1109/ICSE.2013.6606623.

[6] R. Just, D. Jalali, and M. D. Ernst, “Defects4J,” ISSTA, 2014, doi:10.1145/2610384.2628055.

[7] S. Mechtaev, J. Yi, and A. Roychoudhury, “Angelix,” ICSE, 2016, doi:10.1145/2884781.2884807.

[8] F. Long and M. Rinard, “Automatic Patch Generation by Learning Correct Code,” POPL, 2016, doi:10.1145/2837614.2837617.

[9] M. Martinez et al., “Automatic Repair of Real Bugs in Java,” Empirical Software Engineering, 2017, doi:10.1007/s10664-016-9470-4.

[10] R. K. Saha et al., “Elixir,” ASE, 2017, doi:10.1109/ASE.2017.8115675.

[11] M. Allamanis et al., “Learning to Represent Programs with Graphs,” ICLR, 2018.

[12] T. Durieux et al., “Empirical Review of Java Program Repair Tools,” ESEC/FSE, 2019, doi:10.1145/3338906.3338911.

[13] T. Lutellier et al., “CoCoNuT,” ISSTA, 2020, doi:10.1145/3395363.3397369.

[14] R. Widyasari et al., “BugsInPy,” ESEC/FSE, 2020, doi:10.1145/3368089.3417943.

[15] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.

[16] D. Guo et al., “GraphCodeBERT,” ICLR, 2021.

[17] S. Yao et al., “ReAct,” ICLR, 2023.

[18] N. Shinn et al., “Reflexion,” NeurIPS, 2023, doi:10.52202/075280-0377.

[19] C. S. Xia et al., “Automated Program Repair in the Era of Large Pre-trained Language Models,” ICSE, 2023, doi:10.1109/ICSE48619.2023.00129.

[20] C. E. Jimenez et al., “SWE-bench,” ICLR, 2024.

[21] I. Bouzenia et al., “RepairAgent,” arXiv:2403.17134, 2024.

[22] Y. Zhang et al., “AutoCodeRover,” arXiv:2404.05427, 2024.

[23] H. Ruan et al., “SpecRover,” arXiv:2408.02232, 2024.

پیوست الف. اثرانگشت‌های بازتولیدپذیری

نهایی manifest اثر انگشت `8afb87add96e29898ae712f8ef08928b9ba1290a0aabc3ce34bac82adb4b86c2` و برابر import اثر انگشت `80eac046fc614cad1beb7de127ba69aa907b5c588d5409c30e92e5b685c0dc28` است.